Computer Science Department

Database Systems 333

Phase #4

Final Report

Prepared by:

Noor Alhaj -1221543

Shahd Shwekeyeh -1222105

Instructor: Dr.Ihab Alshaer

Section: 7

Date: 18 January 2026

Project Name: Nutri Health

Project Scope and Description

This project aims to develop a database system for Nutri Health Center, a health and nutrition center with multiple branches across Palestine. The center provides two main services: offering health and nutrition consultations to customers and selling health products such as protein supplements and other nutritional items.

The system will manage essential information related to clients, employees, nutrition specialists, branches, and warehouses. It will also support sales, purchases, and inventory management of health products, in addition to handling appointments, diet plans, and payments.

The proposed system aims to improve data organization, support customer follow-up, manage appointments efficiently, and maintain accurate product inventory across all branches.

The goal of the project is to create a database system that can

- Keep track of branch information (locations, managers, employees).
- Manage health products in inventory (products, Quantity, entry dates).
- Track employees, including nutrition specialists and branch managers.
- Manage customers and maintaining customer details, purchase histories, and search capabilities.
- Handle purchases and sales of health products.
- Generate reports, like total products, total sales, profits, top performing products and low-stock alerts.

Expected Entities and Relationships

Entities& Attributes:

- Branches: (Branch_ID, Branch_Name, City, Address, Manager_ID, Phone)
- Employees: (Employee_ID, Employee_Name, Position, Branch_ID, Salary, Hire_Date, Phone, Address)
- Customers: (Customer_ID, Customer_Name, Phone, Address, Join_Date)
- Health Products: (Product_ID, Product_Name, Category, Price, Cost_Price, Expiry_Date, Supplier_ID, Description)
- Inventory: (Inventory_ID, Branch_ID, Product_ID, Quantity, Entry_Date)

- Sales: (Sale_ID, Date, Customer_ID, Employee_ID, Branch_ID, Total_Price)
- Purchases: (Purchase_ID, Date, Supplier_ID, Branch_ID, Total_Cost)
- Supplier: (Supplier_ID, Supplier_Name, Phone, Address, Payment_Terms, Delivery_Time_Days)
- User: (User_ID, User_Name, Employee_ID, User_Password, Role)

Relations

- Each employee works at one branch, the branch has more than one employee.
- Each branch is managed by one employee(manager), and the manager manages one branch.
- Each purchase is received by one branch, and each branch receives more than one purchase.
- Each purchase is provided by one supplier, and the supplier provides more than one purchase.
- Each purchase contains products, and the products contained by more than one purchase.
- Each product is supplied by one supplier, and each supplier supplies more than one product.
- Each product can have multiple inventory records, and each inventory record belongs to exactly one product.
- one branch can have many inventory records, while each inventory record belongs to one branch.
- Each sale is recorded by one branch, and each branch has more than one sale.
- Each sale is handled by one employee, and each employee handles more than one sale.
- Each sale contains products, and each product exists in more than one sale.
- Each sale is made by one customer, and each customer makes more than one sale.
- Each employee has exactly one user account, and the user account for one employee.

Technology Stack

* Database: MySQL

* Backend: Python

* Frontend: HTML, CSS

Technology Justification

We chose MySQL because it's a strong and easy-to-use database that can store a large amount of data in an organized way.

Python was chosen for the backend since it's simple, easy to learn, and has many libraries that help connect the system to the database and perform operations easily.

We used HTML and CSS for the frontend to design nice-looking and easy-to-use web pages, so users can interact with the system in a simple and clear way.

Sample Queries

Branches & Employees

- Retrieve details of all branches located in a specific city.
- Retrieve the names, positions, and salaries of staff members at a specific branch, sorted by staff name.
- Retrieve the name of the manager for each branch, sorted by branch number.
- Retrieve the total number of employees working in each branch.
- Retrieve the total salary expenses per branch.
- Retrieve the number of employees hired during the last year.

Inventory

- Retrieve all health products available at a specific branch, sorted by category.
- Retrieve the total number of products in each category at a specific branch, sorted by supplier.
- Retrieve all products that are currently below the minimum stock level in any branch.

Sales & Purchases

- Retrieve the total sales for each branch in a specific period.
- Retrieve all sales made by a specific branch, sorted by sale date.
- Retrieve total purchases and sales per branch, sorted by branch name.
- Retrieve sales handled by a specific employee within a selected date range.
- Retrieve all purchases made from each supplier during the last quarter.

Customers

- Retrieve a list of inactive customers (no purchases in the last 6 months).
- Retrieve the number of consultations each customer received from nutrition **specialists.**
- Retrieve the product categories most frequently purchased by each customer.

Reports & Analytics

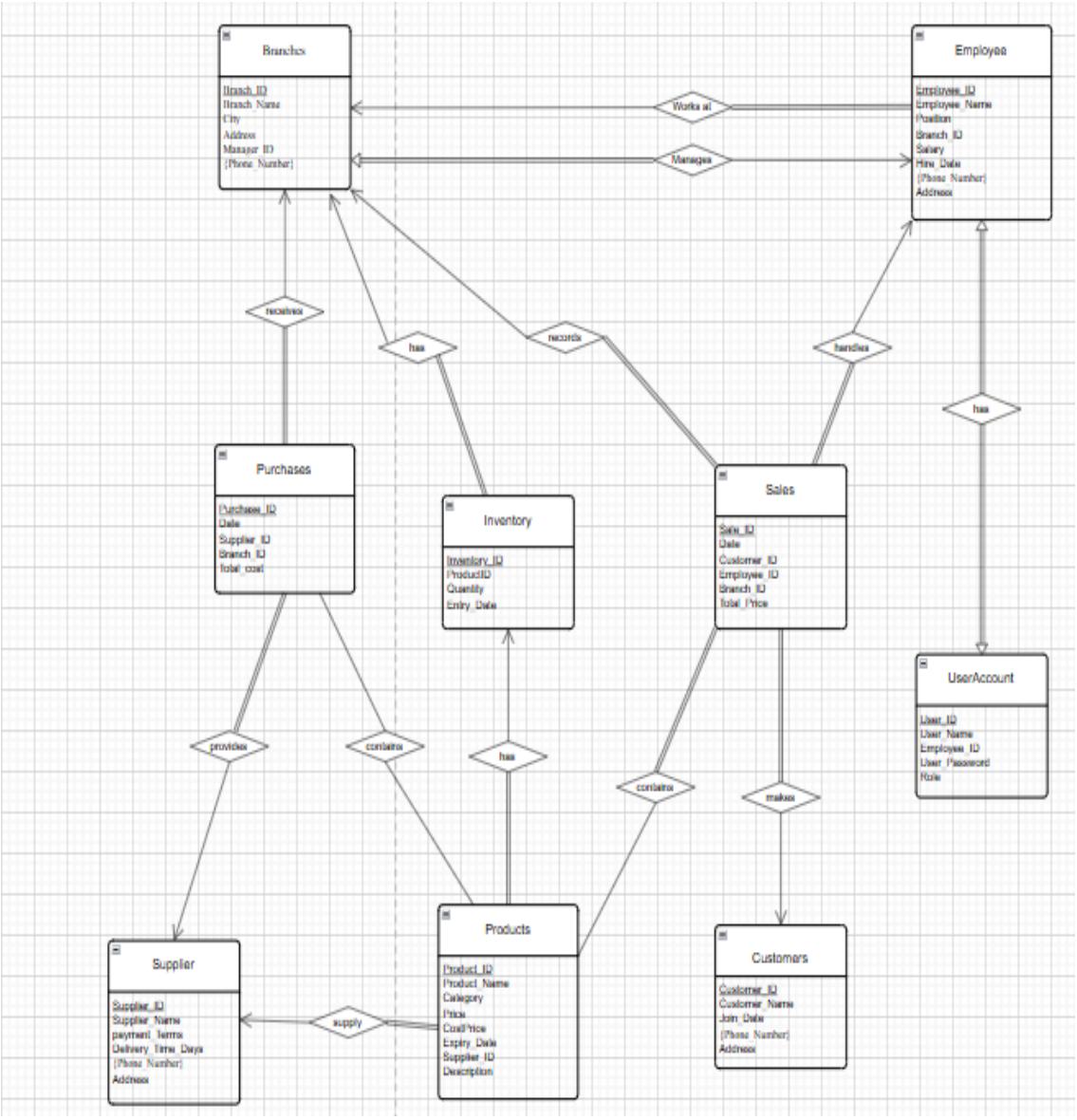
- Retrieve the top 10 best-selling health products across all branches.
- Retrieve the most popular health products sold in each branch.

- Retrieve the profit margin for each product.
- Retrieve total revenue and total expenses for each branch.
- Retrieve a summary of sales, purchases, and profit per branch for management

Reporting

- Retrieve all inventory transfers and stock updates per product, sorted by update date.

ER Diagram



Queries Description and Results

This section explains the SQL queries used in the Nutri Health Center database system. Each query was designed to support management, inventory control, sales analysis, and customer tracking.

A. BRANCHES & EMPLOYEES QUERIES

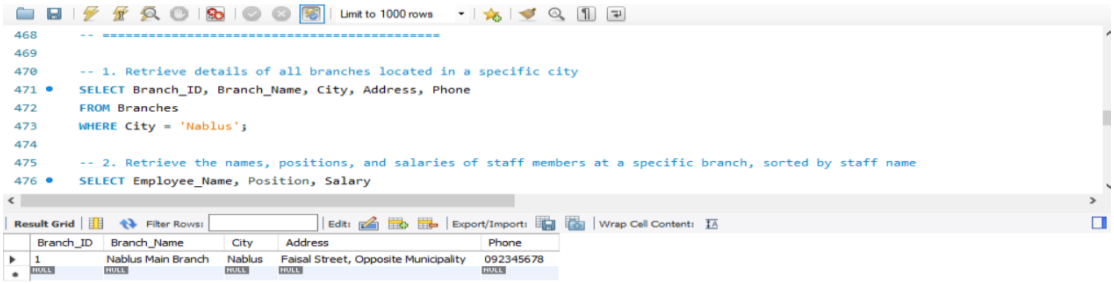
1. Retrieve details of all branches located in a specific city

```
SELECT Branch_ID, Branch_Name, City, Address, Phone
FROM Branches
WHERE City = 'Nablus';
```

Description:
This query retrieves all branches that are located in a specific city (for example, **Nablus**).

Result:
It returns the branch ID, branch name, city, address, and phone number.

Purpose:
It helps users or management find branches available in a certain city.



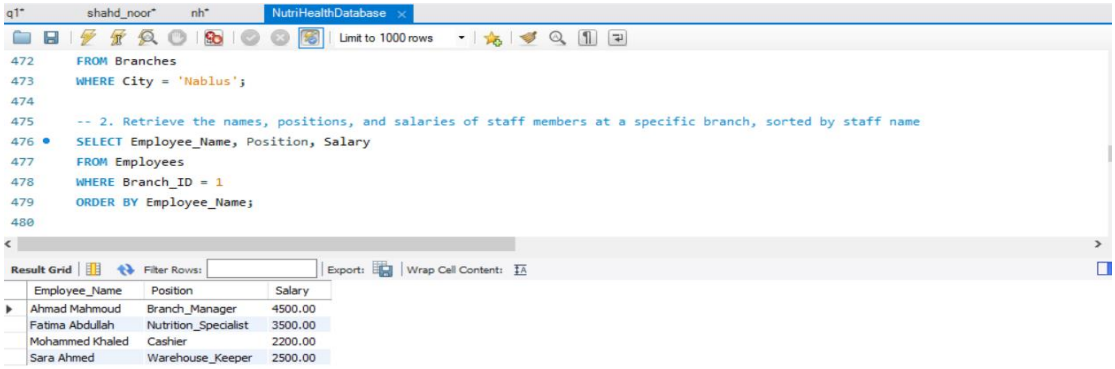
2. Retrieve employees in a specific branch

```
SELECT Employee_Name, Position, Salary
FROM Employees
WHERE Branch_ID = 1
ORDER BY Employee_Name;
```

Description:
This query retrieves the names, positions, and salaries of employees working in a specific branch and sorts them alphabetically.

Result:
A list of employees for the selected branch.

Purpose:
It helps in understanding staff distribution within each branch.



3. Retrieve the manager of each branch

```
SELECT
    b.Branch_ID,
    b.Branch_Name,
    e.Employee_Name AS Manager_Name
FROM Branches b
LEFT JOIN Employees e ON b.Manager_ID = e.Employee_ID
ORDER BY b.Branch_ID;
```

Description:
This query retrieves the name of the manager for each branch by joining the Branches and Employees tables.

How JOIN works: Matches Manager_ID in Branches with Employee_ID in Employees table.

Result:
Each branch with its corresponding manager name.

Purpose:
It shows the management structure of the organization.

```sql
-- 3. Retrieve the name of the manager for each branch, sorted by branch number
SELECT
    b.Branch_ID,
    b.Branch_Name,
    e.Employee_Name AS Manager_Name
FROM Branches b
LEFT JOIN Employees e ON b.Manager_ID = e.Employee_ID
ORDER BY b.Branch_ID;
```

Branch_ID	Branch_Name	Manager_Name
1	Nablus Main Branch	Ahmad Mahmoud
2	Ramallah Branch	Khaled Youssef
3	Hebron Branch	Mariam Issa
4	Jenin Branch	Omar Fares
5	Tulkarm Branch	Tariq Salem

4. Retrieve total number of employees per branch

```sql
SELECT
    b.Branch_Name,
    COUNT(e.Employee_ID) AS Total_Employees
FROM Branches b
LEFT JOIN Employees e ON b.Branch_ID = e.Branch_ID
GROUP BY b.Branch_ID, b.Branch_Name
ORDER BY b.Branch_Name;
```

Description:
This query counts the number of employees working in each branch.

COUNT() counts rows, GROUP BY groups results by branch.

Result:
Branch name with total employee count.

Purpose:
Useful for workforce analysis and branch capacity planning.

```sql
-- 4. Retrieve the total number of employees working in each branch
SELECT
    b.Branch_Name,
    COUNT(e.Employee_ID) AS Total_Employees
FROM Branches b
LEFT JOIN Employees e ON b.Branch_ID = e.Branch_ID
GROUP BY b.Branch_ID, b.Branch_Name
ORDER BY b.Branch_Name;
```

Branch_Name	Total_Employees
Hebron Branch	3
Jenin Branch	2
Nablus Main Branch	4
Ramallah Branch	3
Tulkarm Branch	2

5. Retrieve total salary expenses per branch

```sql
SELECT
```

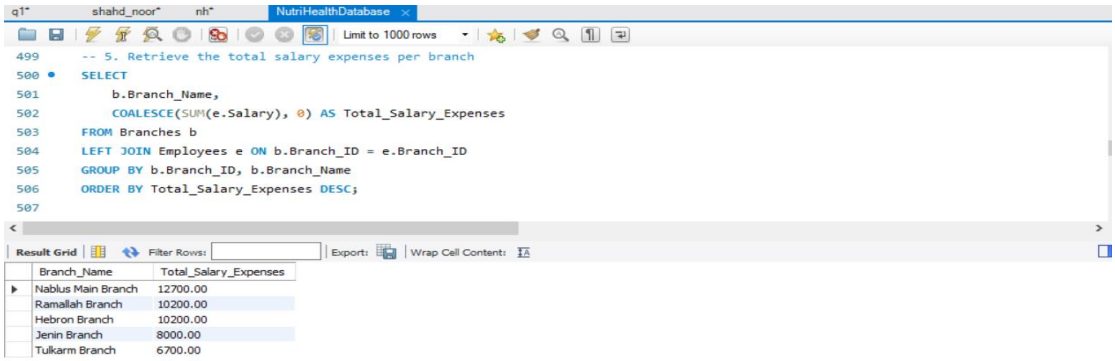
```
    b.Branch_Name,
    COALESCE(SUM(e.Salary), 0) AS Total_Salary_Expenses
FROM Branches b
LEFT JOIN Employees e ON b.Branch_ID = e.Branch_ID
GROUP BY b.Branch_ID, b.Branch_Name
ORDER BY Total_Salary_Expenses DESC;
```

Description:
This query calculates the total salary cost for employees in each branch.

COALESCE(SUM(e.Salary), 0) means "if no employees, show 0 instead of NULL".

Result:
Total salary expenses for each branch.

Purpose:
Helps management analyze operational costs.



6. Retrieve employees hired during the last year

```
SELECT COUNT(*) AS Employees_Hired_Last_Year
FROM Employees
WHERE Hire_Date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR);
```

Description:
This query counts how many employees were hired within the last year.

DATE_SUB(CURDATE(), INTERVAL 1 YEAR) calculates "1 year ago from today".

Result:
A single value representing the number of new employees.

Purpose:
Used to track recruitment and company growth.

```
ORDER BY Total_Salary_Expenses DESC;

-- 6. Retrieve the number of employees hired during the last year
SELECT COUNT(*) AS Employees_Hired_Last_Year
FROM Employees
WHERE Hire_Date >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR);

-- ==========================================
-- Inventory
```

Employees_Hired_Last_Year
14

B. INVENTORY QUERIES

7. Retrieve available products in a specific branch

```
SELECT
    hp.Product_Name,
    hp.Category,
    i.Quantity,
    hp.Price
FROM Inventory i
JOIN Health_Products hp ON i.Product_ID = hp.Product_ID
WHERE i.Branch_ID = 1 AND i.Quantity > 0
ORDER BY hp.Category, hp.Product_Name;
```

Description:
This query retrieves all health products available in a specific branch, including quantity and price.

Result:
A list of products that are currently in stock.

Purpose:
Helps staff know which products are available for sale.

```
-- 7. Retrieve all health products available at a specific branch, sorted by category
SELECT
    hp.Product_Name,
    hp.Category,
    i.Quantity,
    hp.Price
FROM Inventory i
JOIN Health_Products hp ON i.Product_ID = hp.Product_ID
WHERE i.Branch_ID = 1 AND i.Quantity > 0
ORDER BY hp.Category, hp.Product_Name;
```

Product_Name	Category	Quantity	Price
BCAA 2:1:1	Amino Acids	40	95.00
Omega-3 Fish Oil	Omega	45	85.00
Energy Blast Pre-Workout	Pre-Workout	35	120.00
Casein Protein Night	Protein	30	150.00
Plant Protein Blend	Protein	25	140.00
Whey Protein Gold	Protein	50	180.00
Multivitamin Daily	Vitamins	60	70.00
Vitamin C 1000mg	Vitamins	100	45.00

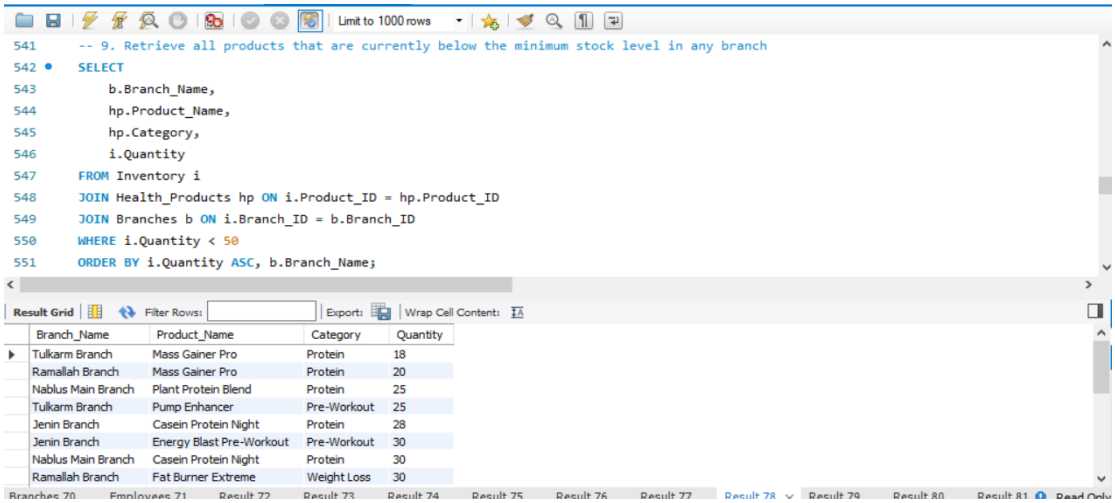
9. Retrieve low-stock products

```
SELECT
    b.Branch_Name,
    hp.Product_Name,
    hp.Category,
    i.Quantity
FROM Inventory i
JOIN Health_Products hp ON i.Product_ID = hp.Product_ID
JOIN Branches b ON i.Branch_ID = b.Branch_ID
WHERE i.Quantity < 50
ORDER BY i.Quantity ASC, b.Branch_Name;
```

Description:
This query retrieves products whose quantity is below a certain threshold (less than 50 units).

Result:
A list of low-stock products with branch names.

Purpose:
Helps management restock products before they run out.



Branch_Name	Product_Name	Category	Quantity
Tulkarm Branch	Mass Gainer Pro	Protein	18
Ramallah Branch	Mass Gainer Pro	Protein	20
Nablus Main Branch	Plant Protein Blend	Protein	25
Tulkarm Branch	Pump Enhancer	Pre-Workout	25
Jenin Branch	Casein Protein Night	Protein	28
Jenin Branch	Energy Blast Pre-Workout	Pre-Workout	30
Nablus Main Branch	Casein Protein Night	Protein	30
Ramallah Branch	Fat Burner Extreme	Weight Loss	30

C. SALES & PURCHASES QUERIES

10.Retrieve total sales per branch within a period

```
SELECT
    b.Branch_Name,
```

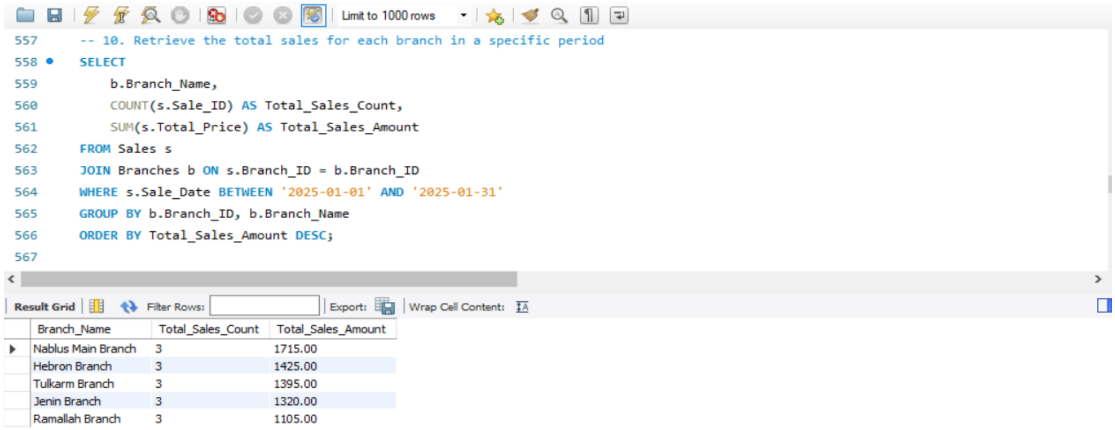
```
    COUNT(s.Sale_ID) AS Total_Sales_Count,
    SUM(s.Total_Price) AS Total_Sales_Amount
FROM Sales s
JOIN Branches b ON s.Branch_ID = b.Branch_ID
WHERE s.Sale_Date BETWEEN '2025-01-01' AND '2025-01-31'
GROUP BY b.Branch_ID, b.Branch_Name
ORDER BY Total_Sales_Amount DESC;
```

Description:
This query retrieves the total number of sales and total sales amount for each branch within a specific date range.

BETWEEN: Filters dates in a range (month of January).

Result:
Sales count and total revenue per branch.

Purpose:
Used to evaluate branch performance over time.



```
-- 10. Retrieve the total sales for each branch in a specific period
SELECT
    b.Branch_Name,
    COUNT(s.Sale_ID) AS Total_Sales_Count,
    SUM(s.Total_Price) AS Total_Sales_Amount
FROM Sales s
JOIN Branches b ON s.Branch_ID = b.Branch_ID
WHERE s.Sale_Date BETWEEN '2025-01-01' AND '2025-01-31'
GROUP BY b.Branch_ID, b.Branch_Name
ORDER BY Total_Sales_Amount DESC;
```

Branch_Name	Total_Sales_Count	Total_Sales_Amount
Nablus Main Branch	3	1715.00
Hebron Branch	3	1425.00
Tulkarm Branch	3	1395.00
Jenin Branch	3	1320.00
Ramallah Branch	3	1105.00

---

13. Retrieve sales handled by a specific employee

```
SELECT
    s.Sale_ID,
    s.Sale_Date,
    CONCAT(c.Customer_FirstName, ' ', c.Customer_LastName) AS Customer_Name,
    s.Total_Price
FROM Sales s
JOIN Customers c ON s.Customer_ID = c.Customer_ID
```

WHERE s.Employee_ID = 3

AND s.Sale_Date BETWEEN '2025-01-01' AND '2025-01-31'

ORDER BY s.Sale_Date DESC;

Description:
This query retrieves all sales transactions handled by a specific employee within a selected date range.

Result:
Sales records associated with that employee.

Purpose:
Helps evaluate employee performance.

```
-- 13. Retrieve sales handled by a specific employee within a selected date range
SELECT
    s.Sale_ID,
    s.Sale_Date,
    CONCAT(c.Customer_FirstName, ' ', c.Customer_LastName) AS Customer_Name,
    s.Total_Price
FROM Sales s
JOIN Customers c ON s.Customer_ID = c.Customer_ID
WHERE s.Employee_ID = 3
  AND s.Sale_Date BETWEEN '2025-01-01' AND '2025-01-31'
ORDER BY s.Sale_Date DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Sale_ID	Sale_Date	Customer_Name	Total_Price
11	2025-01-18	Fadi Hasan	730.00
6	2025-01-17	Dina Yousef	625.00
1	2025-01-15	Amer Saleh	360.00

D. Customer Queries

15. Retrieve inactive customers

SELECT

c.Customer_ID,

CONCAT(c.Customer_FirstName, ' ', c.Customer_LastName) AS Customer_Name,

c.Phone,

c.Join_Date,

MAX(s.Sale_Date) AS Last_Purchase_Date,

DATEDIFF(CURDATE(), MAX(s.Sale_Date)) AS Days_Since_Last_Purchase

FROM Customers c

LEFT JOIN Sales s ON c.Customer_ID = s.Customer_ID

GROUP BY c.Customer_ID

HAVING MAX(s.Sale_Date) < DATE_SUB(CURDATE(), INTERVAL 6 MONTH)

OR MAX(s.Sale_Date) IS NULL

ORDER BY Last_Purchase_Date DESC;

Description:
This query retrieves customers who have not made any purchases in the last six months.

- **DATEDIFF() :calculates days between dates**
- **HAVING :filters groups (like WHERE but for GROUP BY results)**

Result:
A list of inactive customers with last purchase date.

Purpose:
Helps in customer retention and marketing strategies.

```
        c.Customer_ID,
        CONCAT(c.Customer_FirstName, ' ', c.Customer_LastName) AS Customer_Name,
        c.Phone,
        c.Join_Date,
        MAX(s.Sale_Date) AS Last_Purchase_Date,
        DATEDIFF(CURDATE(), MAX(s.Sale_Date)) AS Days_Since_Last_Purchase
    FROM Customers c
    LEFT JOIN Sales s ON c.Customer_ID = s.Customer_ID
    GROUP BY c.Customer_ID
    HAVING MAX(s.Sale_Date) < DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
       OR MAX(s.Sale_Date) IS NULL
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Customer_ID	Customer_Name	Phone	Join_Date	Last_Purchase_Date	Days_Since_Last_Purchase
9	Karim Ibrahim	0598999000	2026-01-18	2025-01-18	365
10	Nadia Mansour	0599000111	2026-01-18	2025-01-18	365
11	Fadi Hasan	0597111222	2026-01-18	2025-01-18	365
12	Maya Khader	0598222333	2026-01-18	2025-01-18	365
13	Bashar Omar	0599333444	2026-01-18	2025-01-18	365
14	Suha Zeid	0597444555	2026-01-18	2025-01-18	365
15	Walid Fares	0598555666	2026-01-18	2025-01-18	365
6	Dina Yousef	0598666777	2026-01-18	2025-01-17	366

17. Retrieve most purchased product categories per customer

SELECT

c.Customer_ID,

CONCAT(c.Customer_FirstName, ' ', c.Customer_LastName) AS Customer_Name,

hp.Category,

COUNT(sd.Product_ID) AS Purchase_Count,

SUM(sd.Quantity) AS Total_Quantity

FROM Customers c

JOIN Sales s ON c.Customer_ID = s.Customer_ID

JOIN Sale_Details sd ON s.Sale_ID = sd.Sale_ID

JOIN Health_Products hp ON sd.Product_ID = hp.Product_ID

GROUP BY c.Customer_ID, c.Customer_FirstName, c.Customer_LastName, hp.Category

ORDER BY c.Customer_ID, Purchase_Count DESC;

Description:
This query retrieves the most frequently purchased product categories for each customer.

Result:
Customer-wise category purchase summary.

Purpose:
Useful for personalized marketing and recommendations.

```
        c.Customer_ID,
        CONCAT(c.Customer_FirstName, ' ', c.Customer_LastName) AS Customer_Name,
        hp.Category,
        COUNT(sd.Product_ID) AS Purchase_Count,
        SUM(sd.Quantity) AS Total_Quantity
    FROM Customers c
    JOIN Sales s ON c.Customer_ID = s.Customer_ID
    JOIN Sale_Details sd ON s.Sale_ID = sd.Sale_ID
    JOIN Health_Products hp ON sd.Product_ID = hp.Product_ID
    GROUP BY c.Customer_ID, c.Customer_FirstName, c.Customer_LastName, hp.Category
    ORDER BY c.Customer_ID, Purchase_Count DESC;
```

Result Grid

Customer_ID	Customer_Name	Category	Purchase_Count	Total_Quantity
1	Amer Saleh	Protein	1	2
2	Lina Mohammad	Vitamins	1	3
2	Lina Mohammad	Protein	1	2
3	Hassan Ali	Vitamins	1	5
4	Reem Khalil	Vitamins	1	5
4	Reem Khalil	Pre-Workout	1	1

E. Reports & Analytics

18. Retrieve top 10 best-selling products

```
SELECT
    hp.Product_ID,
    hp.Product_Name,
    hp.Category,
    SUM(sd.Quantity) AS Total_Quantity_Sold,
    SUM(sd.Subtotal) AS Total_Revenue
FROM Sale_Details sd
JOIN Health_Products hp ON sd.Product_ID = hp.Product_ID
GROUP BY hp.Product_ID, hp.Product_Name, hp.Category
ORDER BY Total_Quantity_Sold DESC
LIMIT 10;
```

Description:
This query retrieves the top 10 best-selling health products based on quantity sold.

Result:
Top-performing products with total revenue.

Purpose:
Helps identify high-demand products.

```
-- 18. Retrieve the top 10 best-selling health products across all branches
SELECT
    hp.Product_ID,
    hp.Product_Name,
    hp.Category,
```

Product_ID	Product_Name	Category	Total_Quantity_Sold	Total_Revenue
1	Whey Protein Gold	Protein	14	2520.00
5	Vitamin C 1000mg	Vitamins	13	585.00
7	Multivitamin Daily	Vitamins	11	770.00
6	Vitamin D3 5000IU	Vitamins	9	495.00
8	B-Complex Plus	Vitamins	5	250.00
9	Calcium Magnesium Zinc	Minerals	3	180.00
11	Omega-3 Fish Oil	Omega	3	255.00
15	BCAA 2:1:1	Amino Acids	3	285.00
2	Casein Protein Night	Protein	2	300.00
4	Mass Gainer Pro	Protein	2	440.00

19. Retrieve most popular products per branch

```
SELECT
    b.Branch_ID,
    b.Branch_Name,
    hp.Product_Name,
    hp.Category,
    SUM(sd.Quantity) AS Total_Sold,
    SUM(sd.Subtotal) AS Total_Revenue
FROM Sales s
JOIN Sale_Details sd ON s.Sale_ID = sd.Sale_ID
JOIN Health_Products hp ON sd.Product_ID = hp.Product_ID
JOIN Branches b ON s.Branch_ID = b.Branch_ID
GROUP BY b.Branch_ID, b.Branch_Name, hp.Product_ID, hp.Product_Name, hp.Category
ORDER BY b.Branch_Name, Total_Sold DESC;
```

Description:
This query retrieves the most popular products sold in each branch.

Result:
Branch-wise product performance.

Purpose:
Helps compare branch sales behavior.

```
        SUM(sd.Quantity) AS Total_Quantity_Sold,
        SUM(sd.Subtotal) AS Total_Revenue
    FROM Sale_Details sd
    JOIN Health_Products hp ON sd.Product_ID = hp.Product_ID
    GROUP BY hp.Product_ID, hp.Product_Name, hp.Category
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Branch_ID	Branch_Name	Product_Name	Category	Total_Sold	Total_Revenue
3	Hebron Branch	Vitamin D3 5000IU	Vitamins	5	275.00
3	Hebron Branch	Whey Protein Gold	Protein	5	900.00
3	Hebron Branch	Calcium Magnesium Zinc	Minerals	3	180.00
3	Hebron Branch	Multivitamin Daily	Vitamins	1	70.00
4	Jenin Branch	Multivitamin Daily	Vitamins	5	350.00
4	Jenin Branch	Vitamin D3 5000IU	Vitamins	4	220.00
4	Jenin Branch	BCAA 2:1:1	Amino Acids	3	285.00
4	Jenin Branch	Vitamin C 1000mg	Vitamins	2	90.00
4	Jenin Branch	L-Glutamine Powder	Amino Acids	2	160.00
4	Jenin Branch	Energy Blast Pre-Workout	Pre-Workout	1	120.00
4	Jenin Branch	Omega-3 Fish Oil	Omega	1	85.00
1	Nablus Main B...	Whey Protein Gold	Protein	5	900.00
1	Nablus Main B...	Mass Gainer Pro	Protein	2	440.00
1	Nablus Main B...	Fat Burner Extreme	Weight Loss	2	260.00
1	Nablus Main B...	Omega-3 Fish Oil	Omega	1	85.00

Important Functions:

- **COUNT():** Count rows
- **SUM():** Add values
- **AVG():** Calculate average
- **MAX()/MIN():** Find highest/lowest
- **CONCAT():** Combine text
- **ROUND():** Round numbers
- **DATEDIFF():** Calculate days between dates

Implementation of Third Normal Form (3NF) in Nutri Health Database

In the Nutri Health Center database project, we applied database normalization rules, especially the **Third Normal Form (3NF)**, to design a clear and efficient database. The main idea of 3NF is that all non-key attributes should depend only on the primary key, and there should be no transitive dependencies between attributes.

To achieve 3NF, we divided the system into several related tables, where each table represents one real-world entity. For example, instead of storing supplier information with every product, we only store the **Supplier_ID** in the **Health_Products** table. The full supplier details, such as name and phone number, are stored in the **Suppliers** table. The same method is used for employees, customers, and branches, where relationships between tables are created using IDs.

This design helps reduce data duplication and keeps the database organized. If any information needs to be updated, such as a supplier's phone number or a branch name, it is updated in only one place. This also helps prevent data inconsistency and makes the system easier to maintain.

In some cases, we stored data that already exists in another table, such as **Unit_Price** in the **Sale_Details** table. This was done on purpose because the unit price represents the actual price at the time of sale. Keeping this value is important for tracking sales history and generating accurate financial reports, even if product prices change later.

Overall, the Nutri Health database follows the rules of **Third Normal Form (3NF)**. The database is well-structured, easy to manage, and suitable for handling multiple branches, products, employees, and customers. This design improves data accuracy and supports future expansion of the system.

Conclusion

In this project, we designed and implemented a complete database system for the Nutri Health Center. The system was built to manage branches, employees, customers, products, inventory, sales, and purchases in an organized and efficient way.

During the design process, we applied proper database design principles and normalized the database up to the Third Normal Form (3NF). This helped us reduce data redundancy, improve data consistency, and make the database easier to maintain and expand in the future. Each entity was stored in a separate table, and relationships between entities were established using primary and foreign keys.

The implemented SQL schema and queries allow the system to generate useful reports such as total sales, inventory status, profits, and employee-related information. These reports can help management make better decisions and monitor the performance of different branches.

Overall, this project helped us understand how to design a real-world database system, apply normalization rules, and write meaningful SQL queries. The Nutri Health database is flexible, well-structured, and meets the project requirements while supporting the daily operations of the center.